

## FINAL PROJECT BRIEF

# JET ENGINE HOSPITAL

## Predicting Failure Before It Happens

A multi-task early-warning system for NASA C-MAPSS turbofan engines

|            |                            |
|------------|----------------------------|
| COURSE     | Machine Learning           |
| UNIVERSITY | Shahid Beheshti University |
| INSTRUCTOR | Dr. Farahani               |
| TERM       | Spring 2026                |
| DUE DATE   | _____                      |



**MISSION** Estimate remaining life, forecast near-term failure, detect anomalies without failure labels, and convert all three signals into a calibrated maintenance recommendation.

## TWO-STAGE PROGRESSION

|                                                                      |                                                                                   |                                                               |
|----------------------------------------------------------------------|-----------------------------------------------------------------------------------|---------------------------------------------------------------|
| <b>01</b><br>FOUNDATION<br><b>FD001</b><br>One condition · one fault | <b>02</b><br>CHALLENGE<br><b>FD002 or FD003</b><br>Condition shift or multi-fault | <b>+</b><br>BONUS<br><b>FD004</b><br>Both challenges together |
|----------------------------------------------------------------------|-----------------------------------------------------------------------------------|---------------------------------------------------------------|

Regression · Classification · Unsupervised Detection · Decision Support

# 1 Overview and Learning Goals

This capstone integrates exploratory analysis, time-series feature engineering, supervised learning, unsupervised anomaly detection, uncertainty estimation, and lightweight deployment. Each engine is a separate multivariate time series. Engines begin with different, unknown levels of normal wear and manufacturing variation; later, a fault develops and propagates toward failure.

The goal is not to chase one headline metric. The goal is to build a defensible early-warning system whose data split, alerts, uncertainty, and maintenance actions can survive technical review.

## BY THE END OF THE PROJECT, YOU SHOULD BE ABLE TO

- construct leakage-safe features and windows from multivariate engine histories;
- estimate Remaining Useful Life (RUL) with a calibrated prediction interval;
- predict failure within 10, 20, and 30 future cycles and compare the horizons;
- detect abnormal behavior without using failure labels;
- measure warning lead time and optimize decisions under asymmetric costs; and
- deploy a compact dashboard that turns model outputs into an auditable action recommendation.



**CORE CONSTRAINT** The train/validation/test split must be made by engine ID before fitting preprocessing, selecting features, creating evaluation windows, tuning thresholds, or calibrating uncertainty. Every window from one engine belongs to exactly one split.

# 2 Problem Formulation: Three Related Tasks

For engine  $i$  at cycle  $t$ , let  $x(i,t)$  contain the three operating settings and 21 sensor measurements observed up to that cycle. The system produces three complementary outputs.

- Regression** — Predict  $RUL(i,t) = T(i) - t$ , where  $T(i)$  is the engine's failure cycle in a run-to-failure trajectory. A piecewise-capped target is allowed, but the cap must be justified and tuned only on training/validation engines.
- Classification** — For each horizon  $h \in \{10, 20, 30\}$ , predict  $y(i,t,h) = 1$  when  $RUL(i,t) \leq h$ . The result is three calibrated probabilities, not one reused binary label.
- Unsupervised detection** — Produce an anomaly score  $a(i,t)$  using no failure label during fitting. Investigate whether the score rises consistently before failure and whether the rise survives operating-condition changes.

These tasks may share a common feature pipeline, but each output must be evaluated with task-appropriate metrics. A multi-task formulation is welcome; it does not remove the requirement to report strong classical baselines.

## 2.1 Final decision output

At inference time, the three model families are combined into one maintenance recommendation. Thresholds must be learned or selected on validation engines using the declared cost policy.

| Recommendation | Typical evidence                                                                        | Required dashboard behavior                         |
|----------------|-----------------------------------------------------------------------------------------|-----------------------------------------------------|
| CONTINUE       | Comfortable lower RUL bound; low calibrated failure probabilities; stable anomaly score | Show green status and next review cycle             |
| INSPECT        | Elevated risk, persistent anomaly, or wide uncertainty                                  | Show amber status, drivers, and inspection priority |
| STOP           | Critical lower RUL bound or high validated near-term failure risk                       | Show red status and the rule that triggered it      |

**! SAFETY NOTE** The table is a decision template, not a universal threshold policy. Your thresholds must be validated and your recommendation logic must remain traceable to RUL, probability, anomaly persistence, and uncertainty.

### 3 Dataset: NASA C-MAPSS

C-MAPSS is a simulation of a realistic commercial turbofan engine and control system. The benchmark contains multiple multivariate run-to-failure trajectories. Training trajectories end at failure; test trajectories stop before failure and are accompanied by one final RUL value per test engine. Sensor noise and unknown initial wear are part of the problem.

#### 3.1 Official benchmark subsets

| Subset | Train engines | Test engines | Operating conditions | Fault modes  | Role in this project      |
|--------|---------------|--------------|----------------------|--------------|---------------------------|
| FD001  | 100           | 100          | 1 (sea level)        | 1: HPC       | Mandatory foundation      |
| FD002  | 260           | 259          | 6                    | 1: HPC       | Condition-shift challenge |
| FD003  | 100           | 100          | 1 (sea level)        | 2: HPC + fan | Multi-fault challenge     |
| FD004  | 248           | 249          | 6                    | 2: HPC + fan | Bonus: combined challenge |

Source: [NASA Open Data Portal — C-MAPSS Jet Engine Simulated Data](#)

#### 3.2 File structure

Each whitespace-separated row is one engine-cycle snapshot with 26 values. Assign column names explicitly when loading the text files; do not treat the first row as a header.

| Columns | Meaning                     | Implementation note                                    |
|---------|-----------------------------|--------------------------------------------------------|
| 1       | engine_id                   | Group key for splitting, labels, windows, and plots    |
| 2       | cycle                       | Time index within one engine                           |
| 3–5     | operational_setting_1 ... 3 | Context variables; especially important in FD002/FD004 |
| 6–26    | sensor_1 ... sensor_21      | Noisy multivariate condition measurements              |

**Illustrative row from FD001:**

```
engine_id=4 | cycle=1 | op=[0.0025, 0.0005, 100.0] | sensors=[518.67, 642.60, ..., 23.3814]
```

#### 3.3 Label construction

- Training RUL: for engine  $i$ , compute  $T(i) = \max(\text{cycle})$  and  $\text{RUL}(i, t) = T(i) - t$ .
- Test RUL: the file `RUL_FD00x.txt` gives the remaining cycles after each test engine's final observed cycle; align by the official engine order and assert that counts match.
- Classification labels: generate three separate columns for  $h = 10, 20, 30$  using only the correctly constructed RUL target.
- Anomaly fitting set: define a defensible healthy region using training engines only; do not use test failure proximity or test RUL values to choose it.

## 4 Two-Stage Project Design

| Stage                | Dataset               | Purpose                                                                            | Minimum outcome                                                                    |
|----------------------|-----------------------|------------------------------------------------------------------------------------|------------------------------------------------------------------------------------|
| Stage 1 — Foundation | FD001                 | Build one reproducible end-to-end pipeline under one condition and one fault mode. | RUL + 10/20/30-cycle risks + anomaly score + common baselines + deployed dashboard |
| Stage 2 — Base grade | Choose FD002 or FD003 | Stress the same method on either multiple conditions or multiple fault modes.      | Fair comparison with FD001; explain what breaks and what must change               |
| Bonus extension      | FD004                 | Manage multiple conditions and multiple fault modes simultaneously.                | Best final comparison using an identical leakage-safe evaluation protocol          |

Stage 2 is not a separate notebook with different conventions. Reuse the same notebook structure, split policy, feature pipeline, metrics, uncertainty method, and dashboard contract so the comparison is meaningful.

## 5 Tasks and Required Evidence

### Task 0 — Reproducible notebook setup

Organize one main notebook so it runs from data loading to final evaluation in a clear top-to-bottom order. Use fixed random seeds, explicit package imports, documented data paths, and deterministic preprocessing wherever possible. Every reported table and figure must be reproducible from the submitted notebook.

| Component              | Minimum requirement                                                                               |
|------------------------|---------------------------------------------------------------------------------------------------|
| Main notebook (.ipynb) | EDA, labels, engine-level split, features, models, calibration, evaluation, and final comparisons |
| Reproducibility        | fixed seeds, install/import cells, documented paths, and a clean top-to-bottom run                |
| Saved artifacts        | preprocessing, fitted models, calibrators, thresholds, and metadata required by the app           |
| App parity             | the app loads the exported artifacts and reproduces the notebook's inference outputs              |

### Task 1 — EDA and data audit

Your analysis must operate at both row level and engine level. At minimum, include:

- engine counts, sequence-length distributions, missing/non-finite values, duplicate (engine\_id, cycle) keys, and cycle continuity;
- variance and near-constant sensor checks performed on training engines only;
- sensor trajectories for several engines, with degradation direction and scale differences discussed;
- operating-setting distributions and, for FD002/FD004, condition clusters or condition-aware normalization;
- class balance for each horizon after the engine-level split; and
- a documented table of every filtering decision and the number of affected engines/rows.

### Task 2 — Time-series feature engineering

Build causal features using observations available at or before cycle  $t$ . Compare a current-cycle baseline with a window-based representation. Candidate features include rolling mean, standard deviation, min/max, slope, exponentially weighted statistics, first differences, sensor ratios, and operating-condition-normalized residuals.

**! CAUSALITY TEST** For any feature at cycle  $t$ , assert that changing rows after  $t$  cannot change that feature. Centered rolling windows, whole-engine normalization, and features computed using the final cycle are leakage.

### Task 3 — RUL regression

Train and compare the following baselines under identical splits and features:

- Linear or Ridge Regression, plus a Polynomial Regression variant;
- Random Forest Regressor;
- Gradient Boosting Regressor; and
- at least one model using explicit window features rather than only the current cycle.

Report MAE, RMSE,  $R^2$ , and an asymmetric maintenance-oriented score or cost. Break results down by early-life, mid-life, and near-failure regions. Include residual plots and at least three engine-level prediction traces.

### Task 4 — Failure-horizon classification

Create separate models or a documented multi-output model for 10-, 20-, and 30-cycle failure horizons. Logistic Regression is the required transparent baseline; compare it with at least one nonlinear classifier.

- Use precision–recall curves and PR-AUC because positive classes can be imbalanced.
- Report precision, recall, F1, PR-AUC, confusion matrices, Brier score, and calibration curves for every horizon.
- Choose decision thresholds on validation engines using the declared asymmetric cost; 0.50 is not automatically optimal.
- Discuss the trade-off: longer horizons create earlier warnings but generally increase ambiguity and false alarms.

### Task 5 — Unsupervised anomaly detection

Implement and compare the following methods using the same leakage-safe feature space where practical:

| Method                   | Required design choice                                 | Evidence to report                                    |
|--------------------------|--------------------------------------------------------|-------------------------------------------------------|
| Isolation Forest         | contamination/threshold and healthy fitting region     | score trajectories and threshold sensitivity          |
| Local Outlier Factor     | novelty=True for held-out inference; neighborhood size | stability across engines and conditions               |
| One-Class SVM            | kernel, $\nu$ , $\gamma$ , and feature scaling         | runtime/scalability and false-alarm behavior          |
| PCA reconstruction error | component count chosen on training/validation data     | reconstruction error by cycle and sensor contribution |

Normalize score direction so larger always means more abnormal. Because raw scales differ across detectors, compare validation percentiles or calibrated ranks rather than unexamined raw magnitudes.

### Task 6 — Early-warning analysis and asymmetric cost

A high row-level classification score is not enough. For each held-out engine, identify the first persistent alert cycle  $\tau(i)$ , define failure cycle  $T(i)$ , and compute lead time  $L(i) = T(i) - \tau(i)$ . A persistent alert should require a documented rule such as  $m$  alerts within the last  $n$  cycles.

| Quantity             | Definition              | Interpretation                                                     |
|----------------------|-------------------------|--------------------------------------------------------------------|
| Lead time            | $L(i) = T(i) - \tau(i)$ | How many cycles of action time the first persistent alert provides |
| Late-warning delay   | $\max(0, h - L(i))$     | Alert arrives inside the desired $h$ -cycle action window          |
| Early-warning burden | $\max(0, L(i) - h)$     | Potential unnecessary inspection or downtime                       |
| Miss rate            | No alert before failure | Safety-critical failure of the warning system                      |

Define and justify a cost such as  $C = c_{\text{miss}} \cdot I(\text{missed}) + c_{\text{late}} \cdot \text{late\_delay} + c_{\text{early}} \cdot \text{early\_burden}$ , with  $c_{\text{miss}} > c_{\text{late}} > c_{\text{early}}$  unless you defend another operational policy. Report both the cost and its components; do not hide missed warnings inside one aggregate number.

**! SIGN CONVENTION** If you also report the PHM/NASA asymmetric exponential score, state the prediction-error sign convention explicitly and verify with a small worked example that late RUL overestimation receives the intended penalty.

## Task 7 — Uncertainty and confidence

Every user-facing alert must include uncertainty. Minimum acceptable evidence:

- RUL: a prediction interval, preferably from split conformal prediction, quantile regression, or a bootstrapped ensemble; report empirical coverage and average width on held-out engines.
- Classification: calibrated probabilities; compare reliability curves and Brier scores before and after calibration.
- Anomaly detection: a normalized score plus a validation-based percentile or threshold margin; do not label an arbitrary raw score as probability.
- Recommendation: display which uncertainty-aware rule fired and avoid STOP when the policy cannot distinguish risk from model uncertainty without justification.

## Task 8 — Dashboard and deployment

Deploy a working interactive dashboard on Hugging Face Spaces using Gradio or Streamlit<sup>1</sup>. A user selects an engine and cycle, then sees the complete evidence chain below.

| Dashboard element | Required content                                                   |
|-------------------|--------------------------------------------------------------------|
| Engine timeline   | selected sensor or health features with current cycle highlighted  |
| RUL card          | point estimate, prediction interval, and units in cycles           |
| Failure risk      | calibrated probabilities for 10, 20, and 30 cycles with thresholds |
| Anomaly card      | normalized score, percentile/threshold, and recent persistence     |
| Action            | CONTINUE / INSPECT / STOP, trigger explanation, and confidence     |
| Model metadata    | dataset, model version, feature window, and last training run      |

Precompute expensive artifacts, keep startup memory modest, and separate the app from training code. The submitted Hugging Face Space must build successfully and remain accessible through the submitted URL. The dashboard should make uncertainty and disagreement visible—for example, high anomaly with low supervised failure risk should not be silently collapsed into a confident answer.

<sup>1</sup> Gradio or Streamlit is used to build the deployed interface, not the predictive model. A future optional LLM may explain outputs, but it must not replace calibrated predictions, uncertainty, or the explicit decision policy.

## 6 Evaluation Protocol

### 6.1 Mandatory engine-level split

Create train, validation, and test engine-ID sets first. Then fit every learned transformation on the training engines only. Validation engines are used for hyperparameters, RUL caps, calibration, persistence rules, and alert thresholds. Test engines are touched once for final reporting.

| Step              | Allowed data       | Leakage check                                        |
|-------------------|--------------------|------------------------------------------------------|
| Split IDs         | engine_id only     | Sets are disjoint and saved to disk                  |
| Fit preprocessing | training engines   | scalers, PCA, condition clusters, selectors fit once |
| Create windows    | within each split  | no window crosses engine or split boundaries         |
| Tune/calibrate    | validation engines | thresholds and intervals never use final test labels |
| Final evaluation  | test engines       | one locked pipeline; no post-test retuning           |

### 6.2 Required model comparison

Use one master results table with identical test engines and confidence intervals. At minimum, compare current-cycle vs. window features, linear vs. tree-based models, the three classification horizons, and all four anomaly detectors. Include model size and inference latency where relevant to deployment.

- Use engine-level bootstrap resampling—not row-level bootstrap—to estimate 95% confidence intervals.
- Include at least two ablations, such as removing operating settings, removing trend features, changing the window length, or omitting calibration.
- Show aggregate metrics and per-engine distributions; a fleet average can hide catastrophic engines.
- For Stage 2, keep the protocol fixed and explain any necessary condition-aware or fault-aware modification.

### 6.3 Minimum metric set

| Task                   | Primary metrics                                     | Required diagnostic                                  |
|------------------------|-----------------------------------------------------|------------------------------------------------------|
| RUL regression         | MAE, RMSE, asymmetric cost/score, 95% CI            | residuals and engine traces; interval coverage/width |
| Failure classification | PR-AUC, precision, recall, F1, Brier, 95% CI        | PR and calibration curves at 10/20/30 cycles         |
| Anomaly detection      | lead-time distribution, false-alert rate, miss rate | score trajectories aligned by cycles-to-failure      |
| Decision policy        | average cost and cost components                    | CONTINUE/INSPECT/STOP confusion by engine            |

## 7 Deliverables and Submission

1. **Notebook** — one complete .ipynb file that runs top-to-bottom and contains data preparation, experiments, evaluation, plots, and the final model comparison.
2. **Application code** — the files required to run the Gradio or Streamlit app, including app code, dependencies, exported model artifacts, calibrators, thresholds, and configuration.
3. **Technical report** — 7–10 pages covering problem framing, data audit, leakage controls, feature engineering, models, uncertainty, results, early-warning costs, error analysis, dashboard logic, and Stage 2 findings.
4. **Hugging Face Space** — a working deployment of the submitted Gradio or Streamlit app and a direct Space URL.

Submission deadline: \_\_\_\_\_

## 8 Bonus: FD004 Combined Challenge

FD004 combines six operating conditions with two fault modes. It is the preferred bonus benchmark because a method must separate operating-context variation from degradation while remaining robust to more than one fault mechanism.

### A STRONG FD004 EXTENSION SHOULD

- reuse the same engine-level split and evaluation contract as Stage 1;
- compare global scaling with condition-aware normalization or operating-regime features;
- test whether one anomaly threshold is stable across regimes;
- analyze performance by operating condition, fault behavior, sequence length, and RUL region;
- compare transfer from FD001/FD002/FD003 against training a dedicated FD004 model; and
- state whether the final dashboard thresholds are global, regime-specific, or dynamically selected.

+

**BONUS PRINCIPLE** A larger model alone is not a bonus. Credit comes from isolating the combined challenge, designing a fair experiment, and explaining why the method generalizes—or fails.

## 9 Practical Pitfalls

| Pitfall                              | Why it is wrong                              | Expected prevention                    |
|--------------------------------------|----------------------------------------------|----------------------------------------|
| Random row split                     | the same engine appears in train and test    | split engine IDs first                 |
| Window before split                  | overlapping histories leak across partitions | window independently inside each split |
| Whole-dataset scaling/PCA            | test distribution influences features        | fit on training engines only           |
| Centered rolling statistics          | future cycles enter current features         | trailing causal windows                |
| Threshold chosen on test             | final performance is optimistically biased   | use validation engines                 |
| Accuracy-only classification         | class imbalance hides failures               | PR analysis and calibrated risk        |
| Raw anomaly score called probability | score has no probabilistic meaning           | report rank/percentile or calibrate    |
| One aggregate metric                 | fleet average hides bad engines              | per-engine plots and distributions     |
| Unexplained STOP rule                | unsafe and unauditable recommendation        | show trigger, uncertainty, and policy  |

## 10 Minimal Interfaces (Illustrative)

```
class FeaturePipeline:
    def fit(self, train_engines): ...
    def transform_engine(self, engine_history, at_cycle=None): ...

class PrognosticsSystem:
    def predict_rul(self, features, interval=True): ...
    def failure_risk(self, features, horizons=(10, 20, 30)): ...
    def anomaly_score(self, features): ...
    def recommend(self, outputs, policy): ...
```



```
class EvaluationSuite:
    def engine_bootstrap(self, predictions, n_boot=1000): ...
    def warning_lead_time(self, timeline, persistence_rule): ...
    def asymmetric_cost(self, warnings, cost_policy): ...
```

## 11 Final Review Checklist

|                          |                                                                                         |
|--------------------------|-----------------------------------------------------------------------------------------|
| <input type="checkbox"/> | Official dataset roles and file structure are documented.                               |
| <input type="checkbox"/> | Engine IDs are disjoint across train, validation, and test.                             |
| <input type="checkbox"/> | No causal feature or window uses future cycles.                                         |
| <input type="checkbox"/> | RUL, 10/20/30-cycle risks, and anomaly scores are all implemented.                      |
| <input type="checkbox"/> | Required classical baselines are compared fairly.                                       |
| <input type="checkbox"/> | Thresholds, calibration, and prediction intervals use validation engines only.          |
| <input type="checkbox"/> | Warning lead time, missed warnings, early burden, and late delay are reported.          |
| <input type="checkbox"/> | Dashboard recommendations show evidence and uncertainty.                                |
| <input type="checkbox"/> | FD002 or FD003 is completed for Stage 2; FD004 is clearly identified as bonus.          |
| <input type="checkbox"/> | The submitted notebook runs top-to-bottom and reproduces the final figures and metrics. |
| <input type="checkbox"/> | The Hugging Face Space is live and matches the submitted app code and model artifacts.  |

## References and Data Attribution

- [1] A. Saxena, K. Goebel, D. Simon, and N. Eklund, “Damage Propagation Modeling for Aircraft Engine Run-to-Failure Simulation,” Proceedings of PHM 2008, Denver, Colorado, 2008.
- [2] A. Saxena and K. Goebel, “Turbofan Engine Degradation Simulation Data Set,” NASA Ames Prognostics Data Repository.
- [3] D. K. Frederick, J. A. DeCastro, and J. S. Litt, User’s Guide for the Commercial Modular Aero-Propulsion System Simulation (C-MAPSS), NASA/TM—2007-215026, 2007.

Official dataset page: [NASA C-MAPSS dataset page](#)